

OPTION & RESULT

I. Option

Generic types

A **generic type** in Rust lets you write **flexible, reusable** code that works with *any* data type — while still keeping it **type-safe** and **compiled**.

Exercice 0: Generic types

Definition

In Rust, **Option** is an **enum** that represents the possibility of a **value being present or absent** — essentially Rust's way of handling **null** or optional values **safely**.

"Maybe there's a value here... maybe not. »

💣 The “billion dollar mistake”

In 2009, computer scientist **Tony Hoare** — who invented the *null reference* in 1965 — famously called it his:

“Billion-dollar mistake” 🧨

Because:

- Null references led to countless bugs, crashes, and security issues.
- Billions in wasted time and broken systems.
- Every language that adopted `null` inherited the risk.



```
enum Option<T> {  
    Some(T),    // contains a value  
    None,       // no value  
}
```




```
let some_number: Option<i32> = Some(42);  
let no_number: Option<i32> = None;
```



```
fn divide(a: i32, b: i32) -> Option<i32> {  
    if b == 0 {  
        None  
    } else {  
        Some(a / b)  
    }  
}
```



```
match divide(10, 2) {  
    Some(result) => println!("Result is {}", result),  
    None => println!("Cannot divide by zero"),  
}
```

Exercise 1: Unwrap & expect

✅ What `.unwrap()` does:

If you have an `Option<T>`, calling `.unwrap()` will:

- ✅ return the value if it's `Some`
- ❌ panic and crash the program if it's `None`

rust

Copy Edit

```
let maybe_value = Some(42);
let value = maybe_value.unwrap(); // ✅ OK

let nothing: Option<i32> = None;
let value = nothing.unwrap(); // ❌ Panics at runtime!
```

✅ What to do instead:

Use pattern matching or combinators like `.map()`, `.unwrap_or()`, `.unwrap_or_else()`, `.and_then()`, etc.

Match example:

rust

Copy Edit

```
match maybe_value {
    Some(val) => println!("Value is {}", val),
    None => println!("No value found"),
}
```

Safe fallback:

rust

Copy Edit

```
let value = maybe_value.unwrap_or(0); // U ↓ 0 if None
```

Used everywhere

1. `Option` is everywhere in Rust

- Used by the standard library constantly.
- Idiomatic way to represent things that *may* be present.
- You'll see it in functions like `.get()`, `.next()`, `.find()`, etc.

2. `Option` is zero-cost

- No runtime overhead compared to using `null` pointers.
- Compiler *optimizes it away* in most cases.
- It's built on top of safe, predictable memory models.

rust

 Copy  Edit

```
let maybe_ref = vec.get(3); // Returns Option<&T>
```

Chaining methods



```
maybe_user
```

```
    .map(|u| u.email)  
    .filter(|email| email.contains("@"))  
    .unwrap_or("no-email@example.com".into());
```


Exercices:
Get methods on Options

II. RESULT

In Rust, **Result** is used to handle errors safely. It's an enum with two outcomes: **Ok(value)** for success, and **Err(error)** for failure.

This forces you to handle both cases **explicitly**, avoiding silent failures or crashes. It replaces exceptions with clear, **compile-checked** logic, making Rust code more reliable and predictable.



```
enum Result<T, E> {  
    Ok(T),    // success, with value T  
    Err(E),   // error, with value E  
}
```

Exercise 5: Result basics


```
fn divide(a: i32, b: i32) -> Result<i32, String> {  
    if b == 0 {  
        Err("cannot divide by zero".into())  
    } else {  
        Ok(a / b)  
    }  
}
```

```
match divide(10, 2) {  
    Ok(result) => println!("Result is {}", result),  
    Err(e) => println!("Error: {}", e),  
}
```

Exercise 6: Divide Result

The ? Operator

4. Ergonomic error handling with ?

The `?` operator lets you bubble up errors without verbose `match` blocks.

rust

 Copy  Edit

```
fn read_file() -> Result<String, std::io::Error> {  
    let content = std::fs::read_to_string("file.txt")?;  
    Ok(content)  
}
```


Exercise 7: Read file Result

Your turn!